

Tafila Technical University

Faculty of Information Technology

Infrastructure Security Using Linux

Deploying and Securing Matomo Web Application on Apache Server

---

Instructor: DR. Yazan Al-sarairah

By:

Student's names: Noor Al-Shater

Student ID:320220605106

1. Introduction:

This report presents the deployment and penetration testing of the Matomo web analytics platform on an Ubuntu Server environment. The objective of this project is to install, configure, and assess the security posture of the web application using common penetration testing tools and techniques. Matomo was selected because it is an open-source, self-hosted analytics platform that provides full control over data and offers a realistic environment for applying Linux-based security and penetration testing concepts.

2. Environment and System Setup:

This section describes the environment configuration and system preparation steps performed before deploying the Matomo web application.

2.1 System Environment Overview

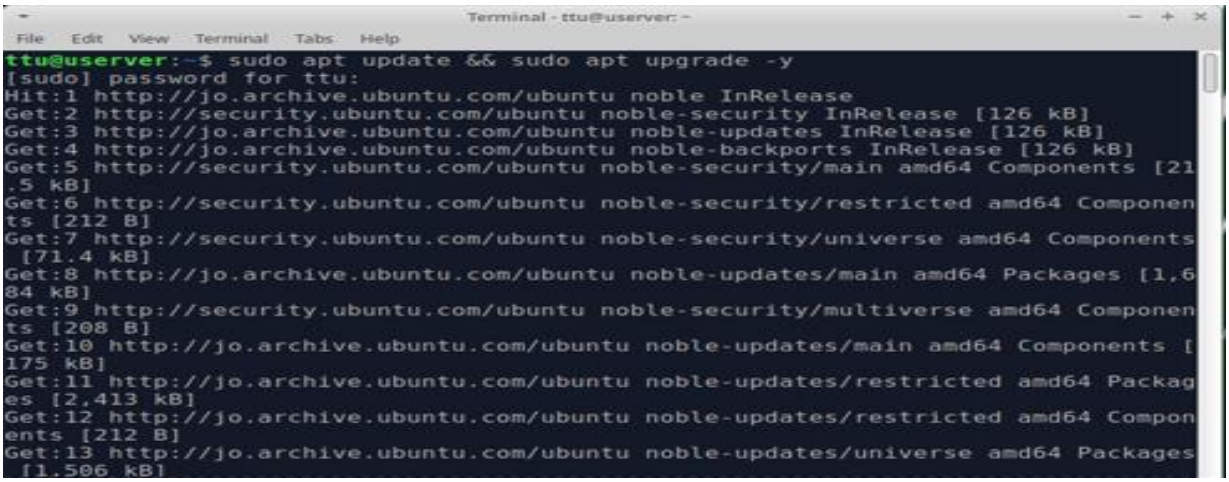
The project was implemented on an Ubuntu Server running inside a virtualized environment. The server was prepared to host a web-based analytics platform using standard open-source components, simulating a real-world production infrastructure.

System Specifications:-

- Operating System: Ubuntu Server 24.04
- Web Server: Apache 2.4.58
- Database Server: MariaDB
- Programming Language: PHP 8.3
- Web Application: Matomo 5.6.2
- Virtualization Platform: Oracle VirtualBox

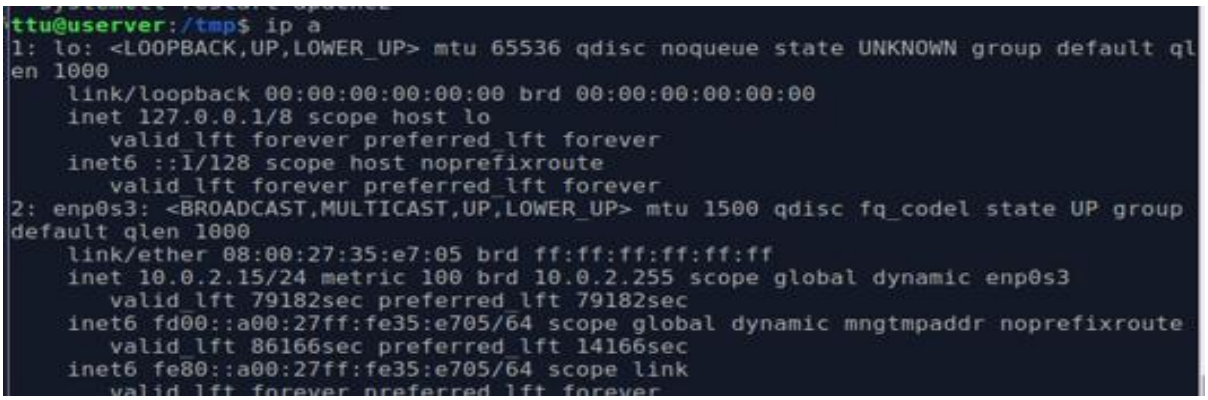
2.2 System Update and Preparation

Before installing any services, the Ubuntu Server was fully updated to ensure system stability and the latest security patches were applied.



2.3 Network Configuration

The network configuration of the Ubuntu Server was verified using the **`ip a`** command.The server was assigned a private IP address (10.0.2.15) through the VirtualBox NAT network adapter.
This IP address was later used to access and test the Matomo web application and to perform penetration testing.

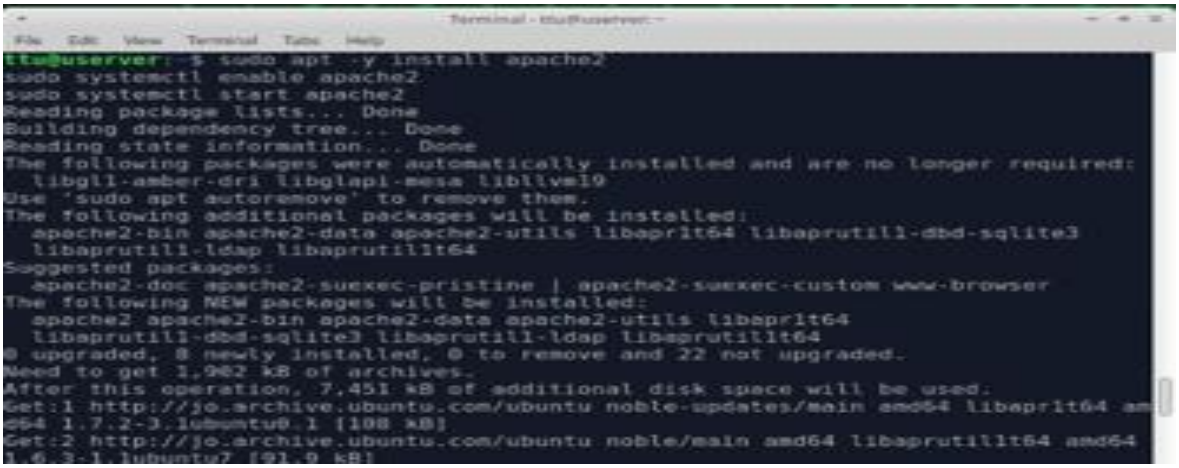


Network interface configuration and assigned IP address of the Ubuntu Server

2.4 Apache Web Server Installation & Configuration

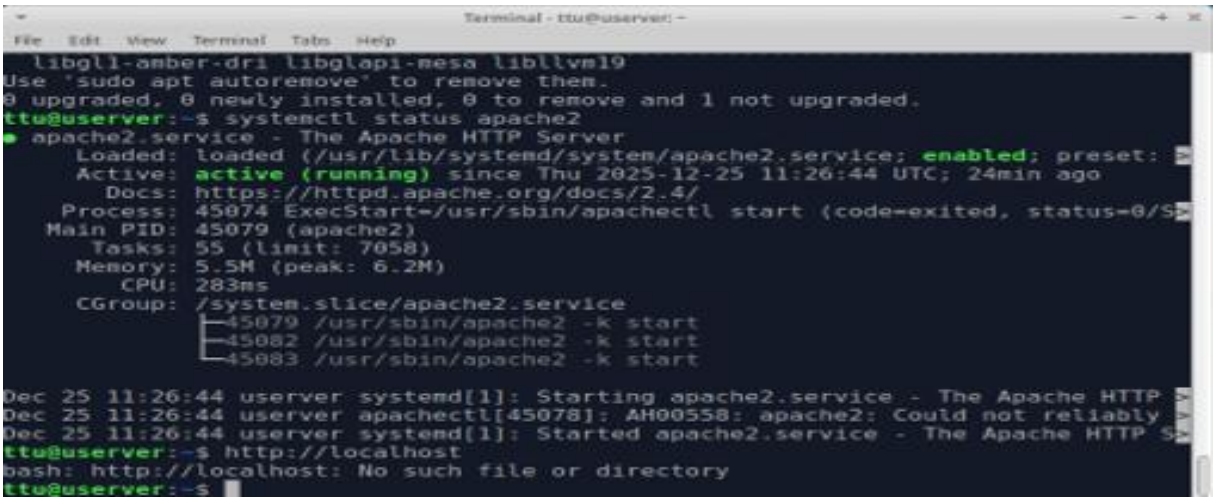
After updating the system, the Apache web server was installed to host the Matomo web application.

Apache was installed, enabled, and started to ensure it runs automatically at system boot and is ready to serve web requests.



2.5 Apache Web Server Status Verification

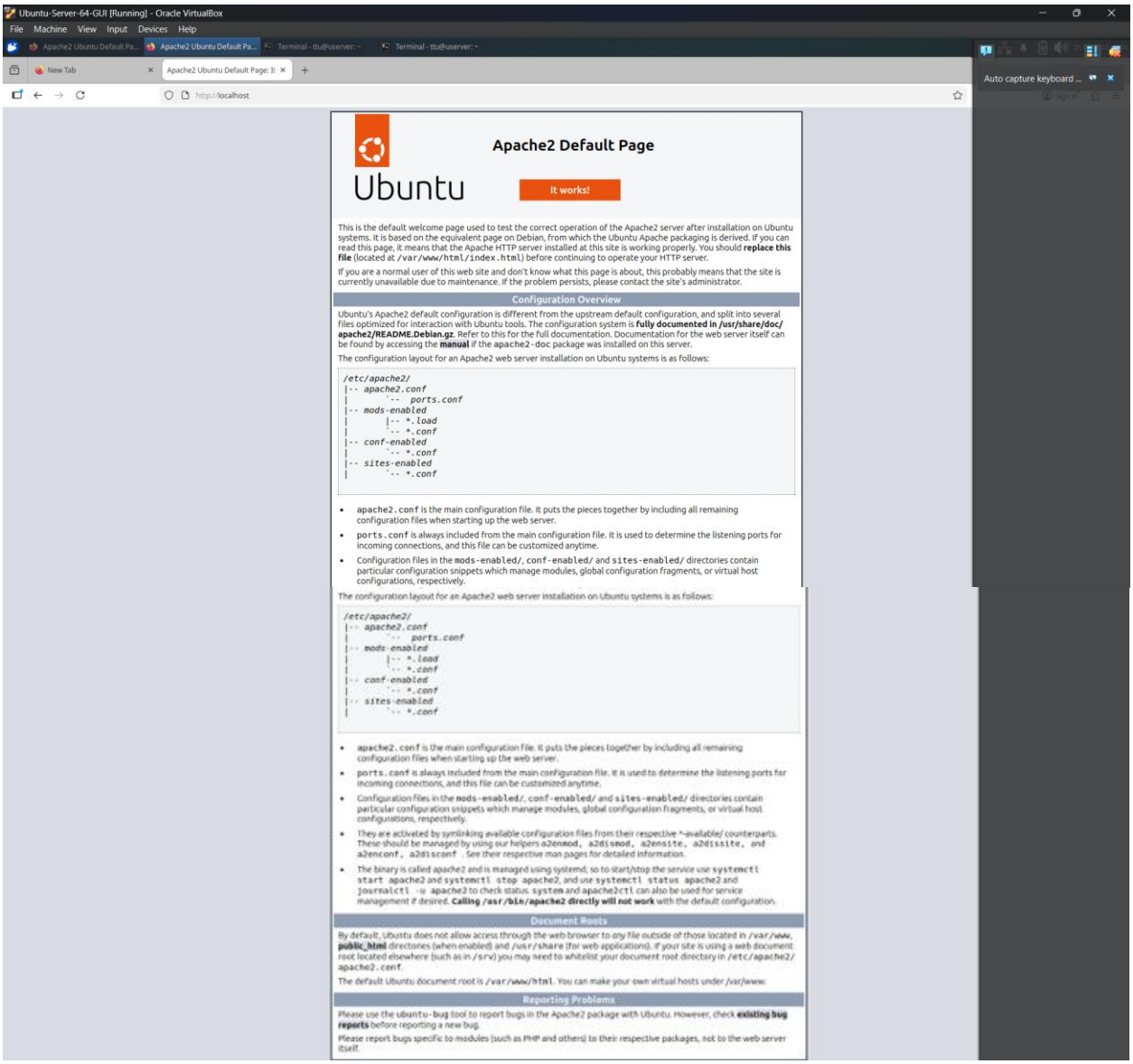
The Apache HTTP server was verified to be running successfully and configured to start automatically on system boot.
The `systemctl status apache2` command confirmed that the service was active and enabled.



This step ensures that the web server is operational and ready to host the Matomo web application.

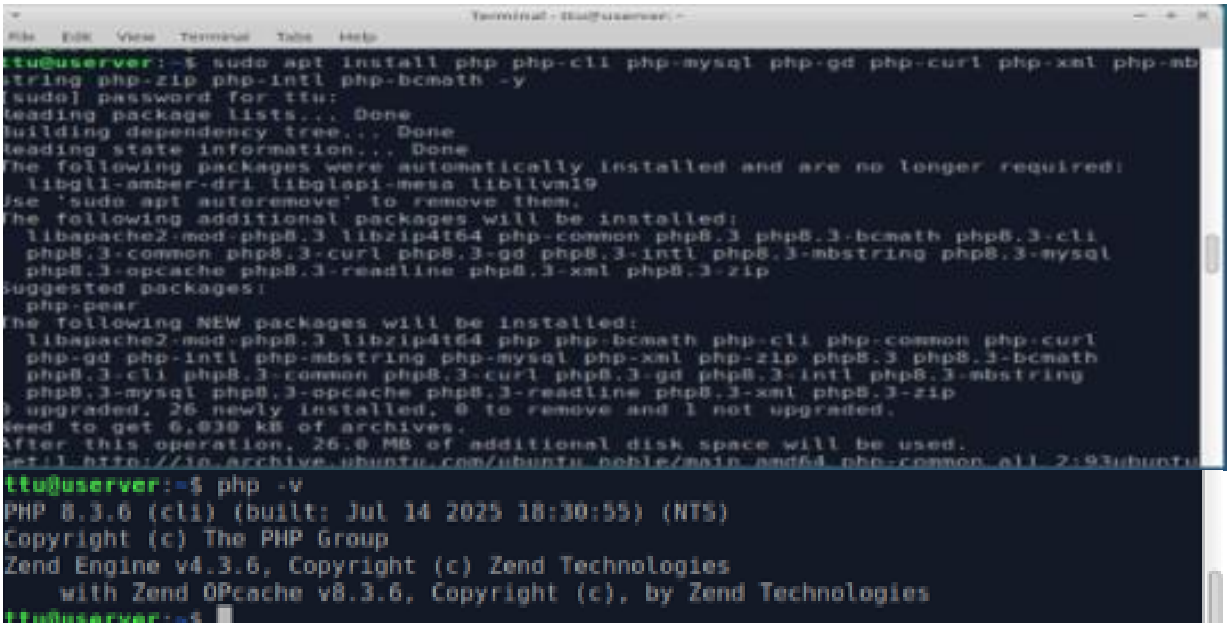
2.6 Apache Web Server Functional Verification (Browser Test)

The Apache2 default web page was successfully accessed through the web browser, confirming that the Apache HTTP server was properly installed, configured, and operational.
This verification demonstrates that the web server is correctly serving HTTP requests and is accessible from the client side.



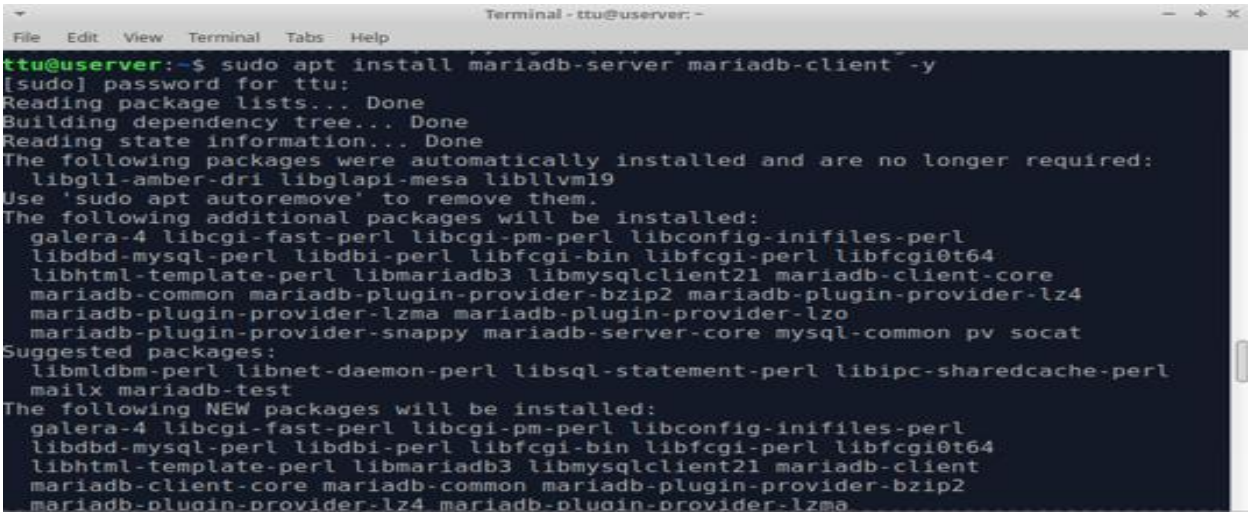
2.7 PHP Installation & Verification

PHP version 8.3.6 was successfully installed and verified, along with the OPcache extension, to support the requirements of the Matomo web analytics platform.



2.8 MariaDB Installation & Configuration

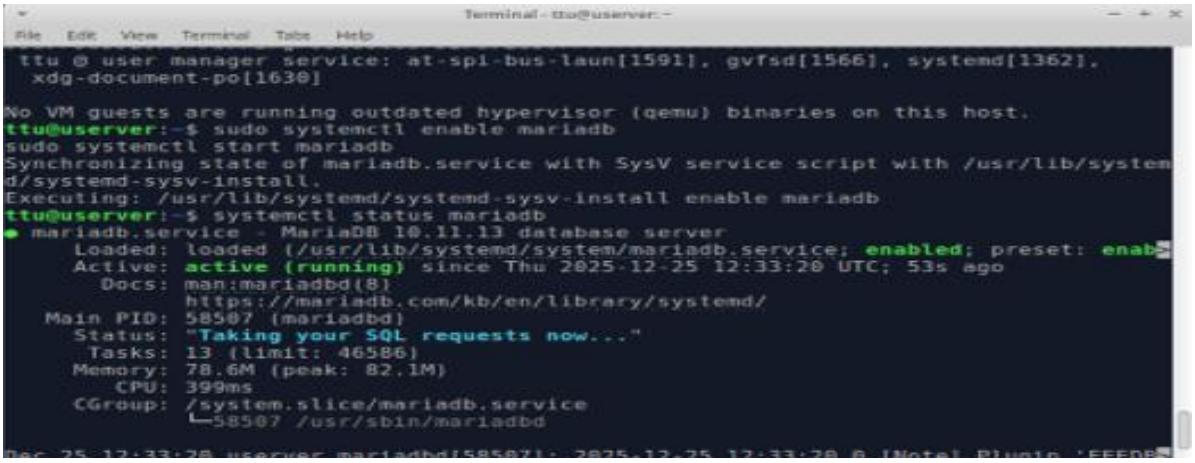
MariaDB server and client packages were successfully installed on the Ubuntu Server to provide the database backend required for the Matomo web analytics platform.



Installation of MariaDB server and client packages on Ubuntu Server

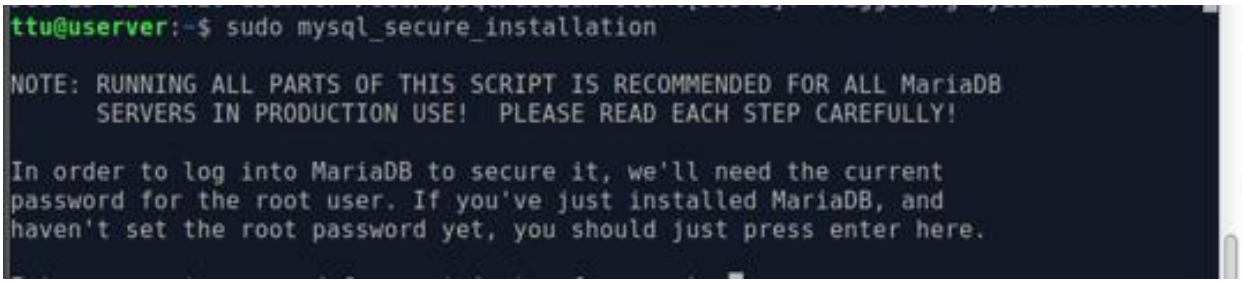
2.9 MariaDB Service Status Verification

The MariaDB database service was verified to be active and running successfully on the Ubuntu Server using the systemctl status command.



2.10 MariaDB Security Hardening

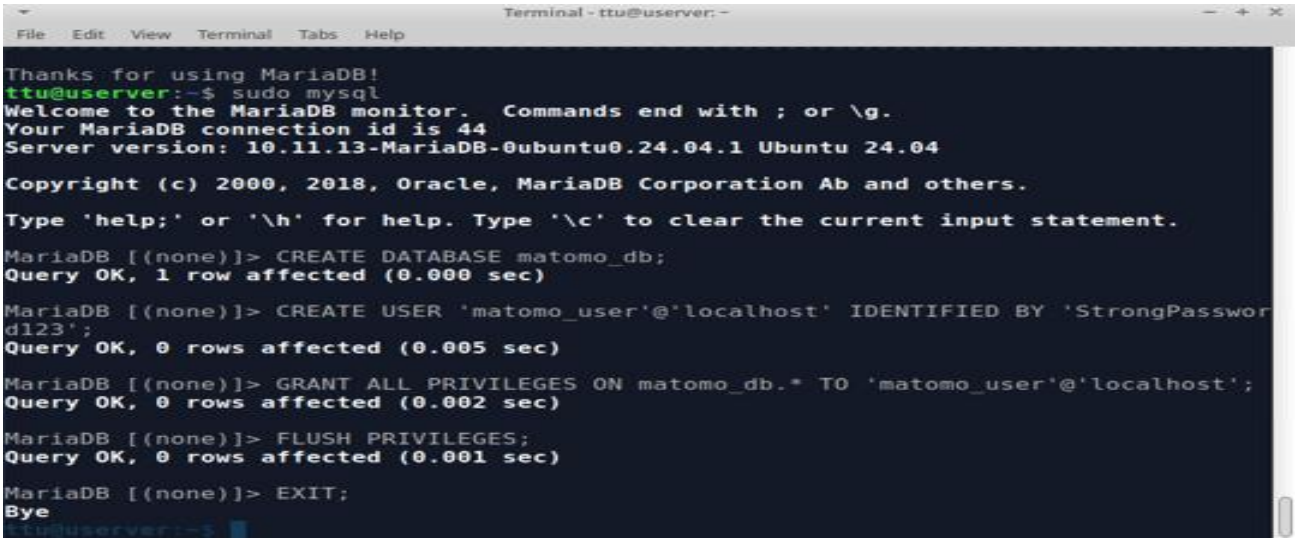
MariaDB security hardening was performed using the `mysql_secure_installation` utility. This process included enabling unix socket authentication, removing anonymous users, disabling remote root login, removing the test database, and reloading privilege tables to enhance database security.



3. Application Deployment (Matomo Installation):

3.1 Matomo Database Configuration

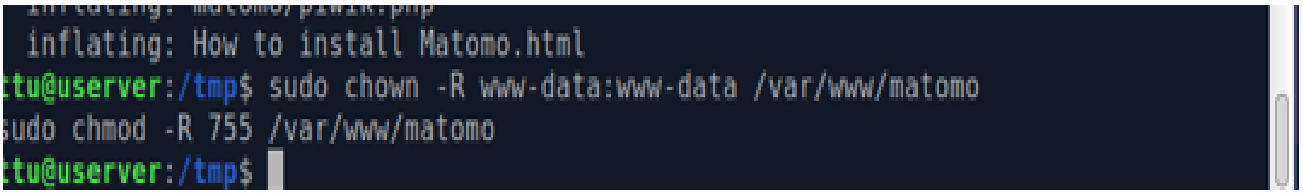
A dedicated database (`matomo_db`) and a restricted database user (`matomo_user`) were created for the Matomo application following the principle of least privilege.
The database user was granted only the necessary permissions required for Matomo operation, enhancing overall database security.



Creation of Matomo database and dedicated database user in MariaDB

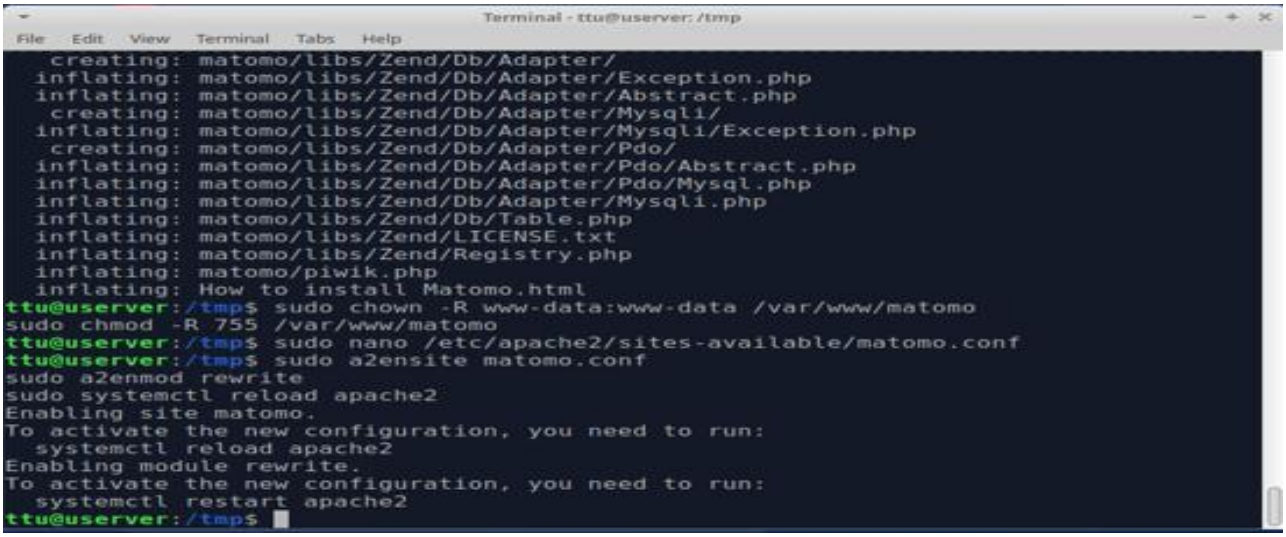
3.2 Matomo Deployment on Apache

The Matomo web analytics platform was successfully extracted and deployed under `/var/www/matomo`.
Proper ownership and permissions were configured to ensure secure access by the Apache web server while preventing unauthorized system-level access.



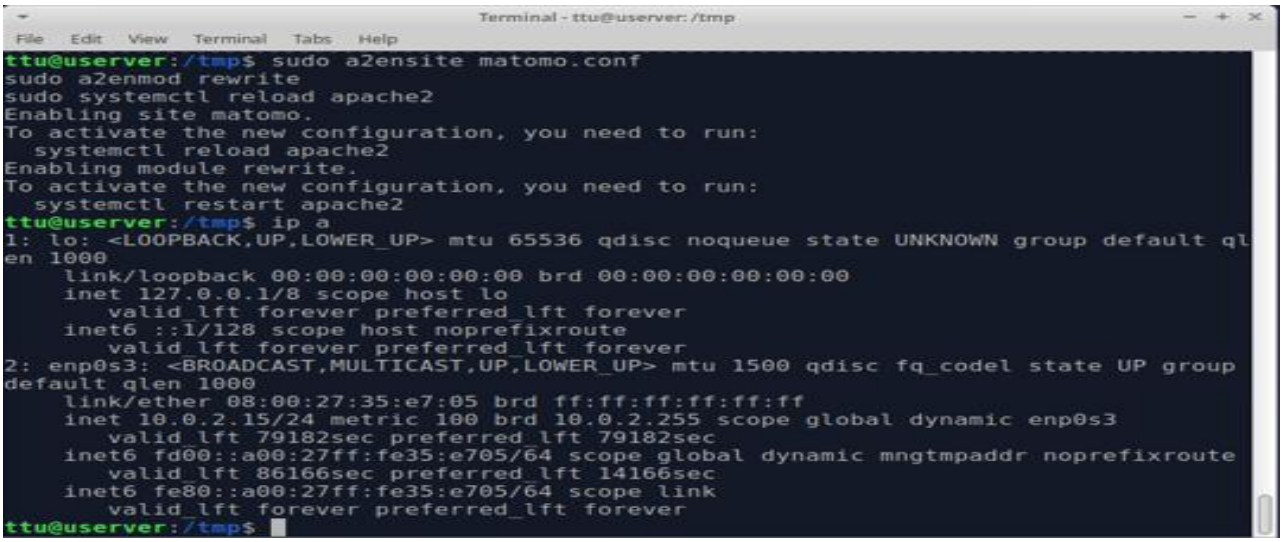
3.3 Apache Virtual Host Configuration

A dedicated Apache Virtual Host was configured and enabled for the Matomo application. The configuration defined a custom domain (`matomo.local`**),** enabled URL rewriting, and ensured proper integration with the Apache web server.



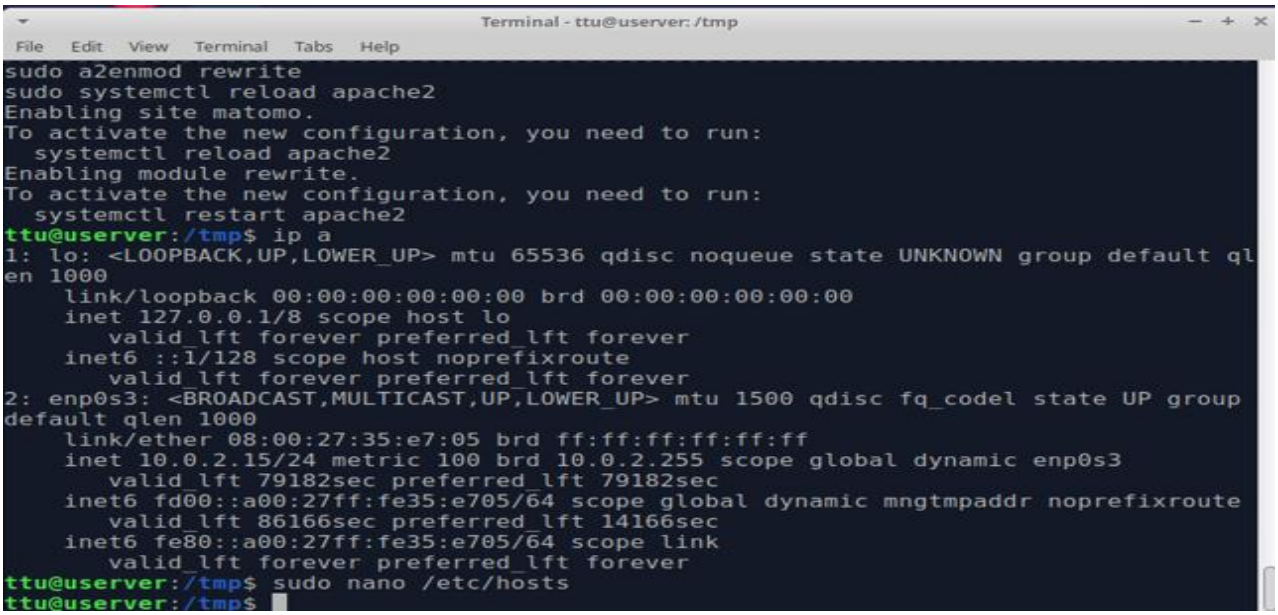
3.4 Apache Virtual Host Activation & Network Verification

The Apache virtual host for the Matomo application was enabled using the `a2ensite` command, and the rewrite module was activated to support clean URLs.
After reloading the Apache service, the network configuration was verified using the `ip a` command, confirming that the server was assigned the IP address **10.0.2.15**, which was used to access the Matomo web application locally.



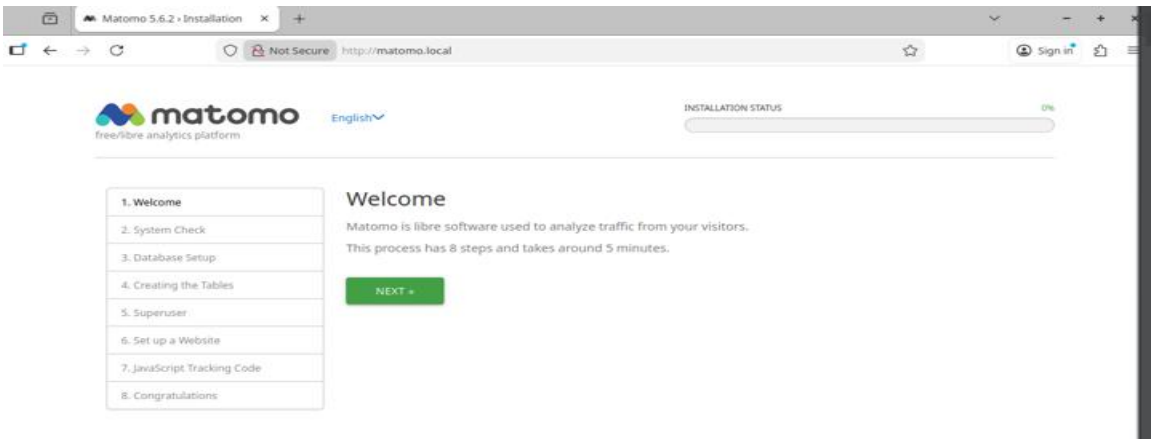
3.5 Local Domain Configuration

The local hosts file was updated to map the domain name **matomo.local** to the server IP address **10.0.2.15**.This configuration enabled accessing the Matomo web application using a local domain name instead of a direct IP address, simulating a real-world web hosting environment.



3.6 Matomo Web Installer – Welcome Page

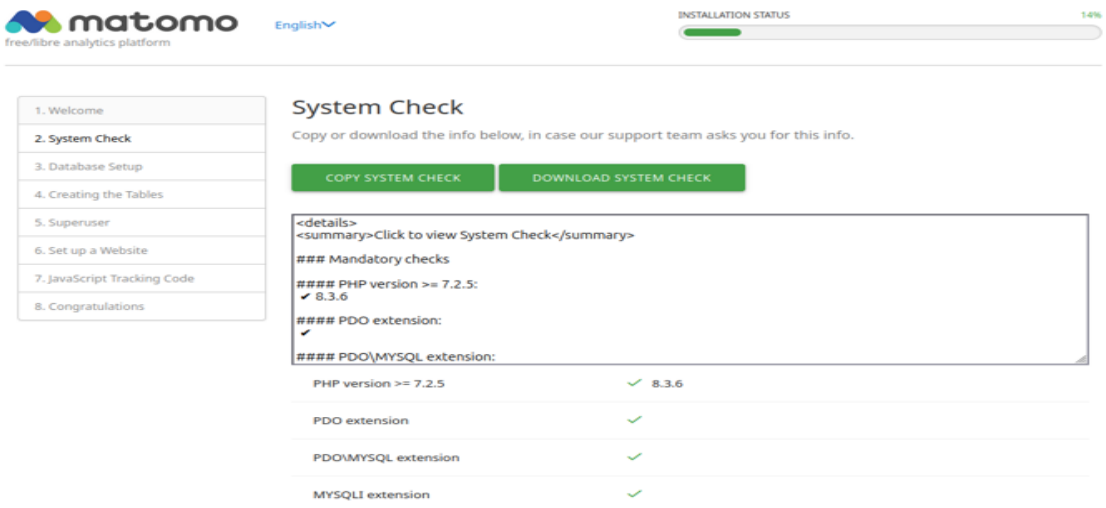
The Matomo web installer was successfully accessed using the local domain name *matomo.local*, confirming that the Matomo application was correctly deployed and reachable through the Apache web server.



Matomo Installation Welcome Page accessed via matomo.local

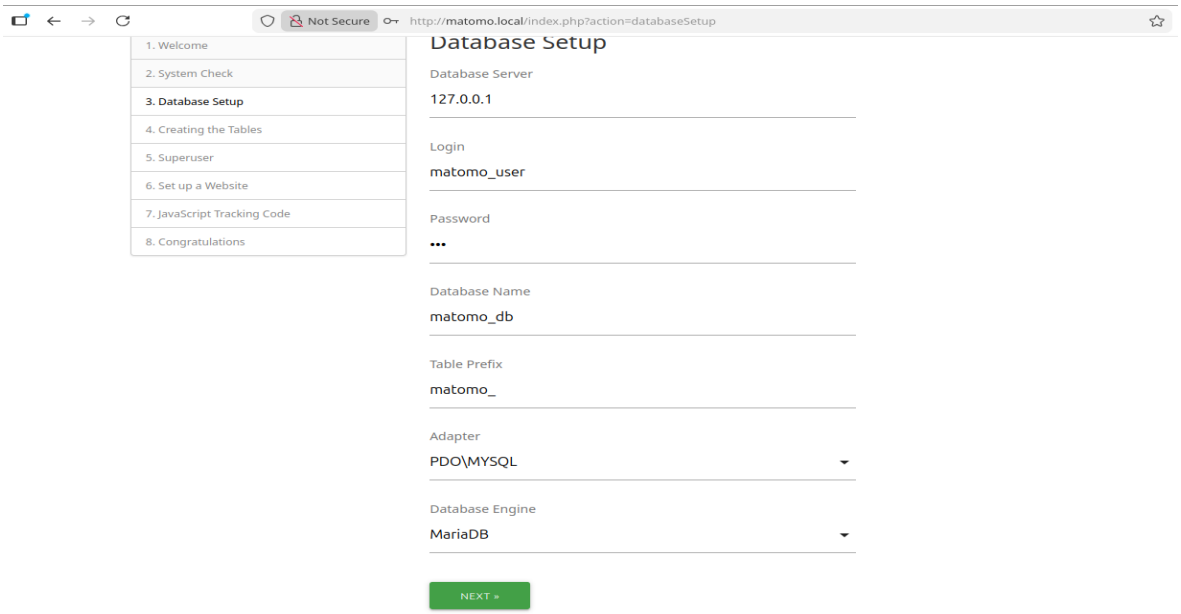
3.7 Matomo System Check

The Matomo system check was completed successfully, confirming that all required PHP versions and extensions were properly installed and correctly configured on the Ubuntu Server environment.



3.8 Matomo Database Configuration

Matomo was configured to connect to a dedicated MariaDB database using a separate database user, following the principle of least privilege.



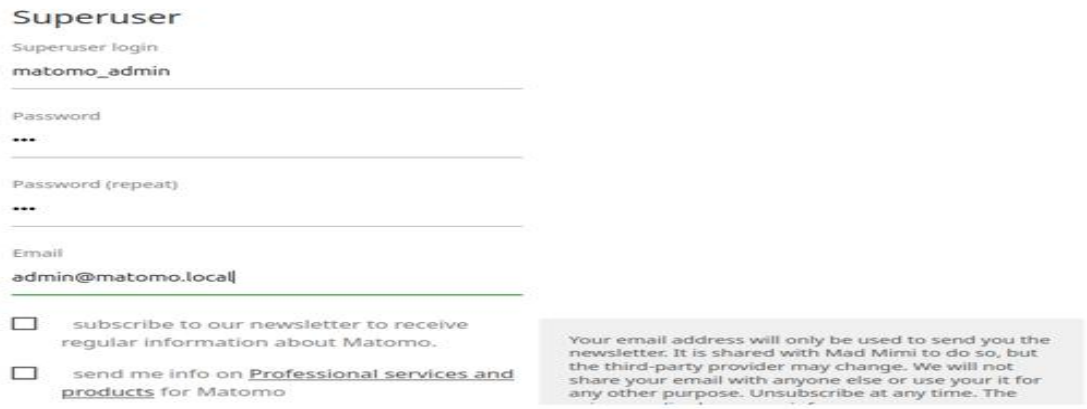
3.9 Database Tables Creation

The Matomo installer successfully created all required database tables without errors.



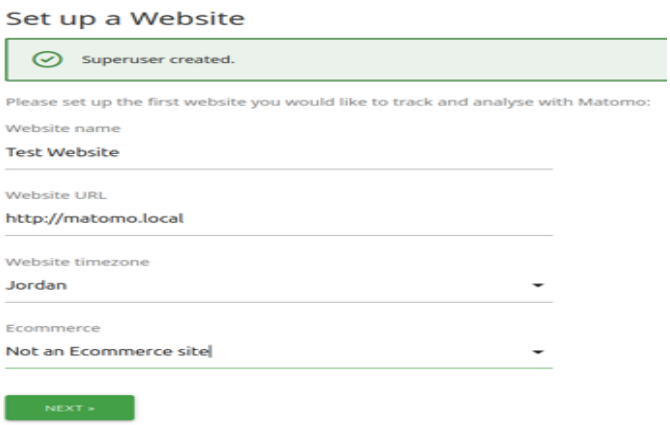
3.10 Matomo Superuser Account Setup

A superuser account was created to manage and administer the Matomo analytics platform.



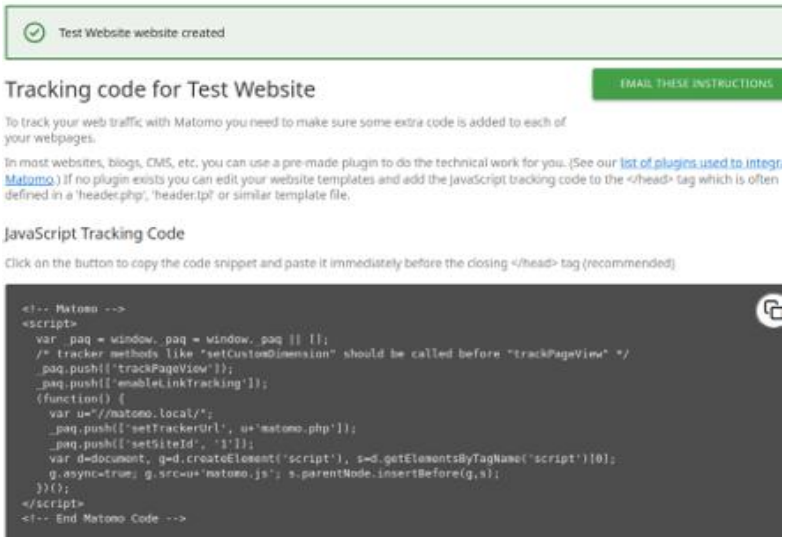
3.11 Website Tracking Configuration

The first website was configured in Matomo, including website name, URL, and timezone settings.



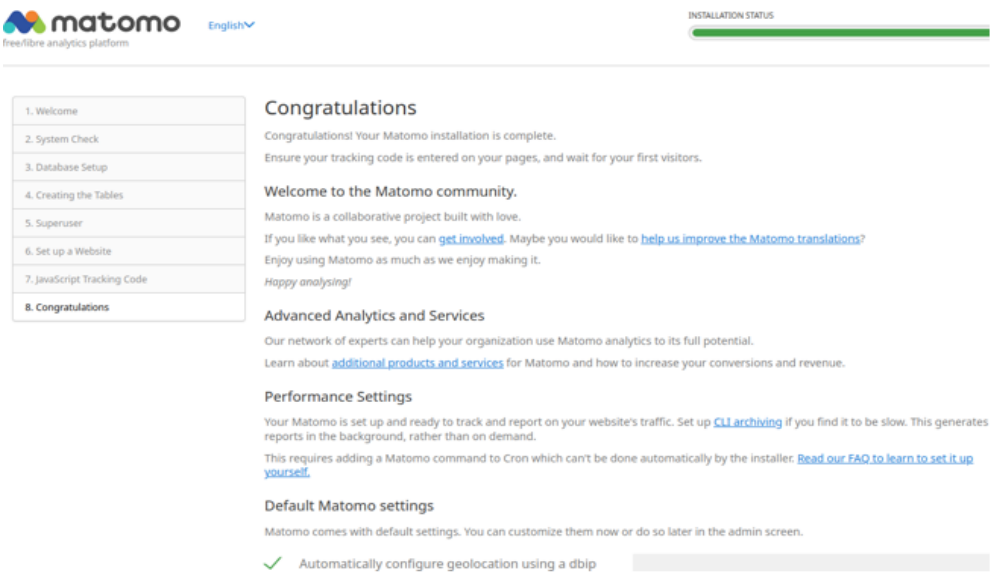
3.12 Matomo Tracking Code Generation

Matomo generated a JavaScript tracking code to be embedded into the website pages for visitor tracking.



3.13 Matomo Installation Completion

The Matomo installation process was completed successfully, and the platform is now ready for analytics and reporting.



4.Penetration Testing & Security Assessment

This section presents the penetration testing activities conducted on the deployed Matomo web application to identify potential security weaknesses.

4.1 Testing Scope & Methodology

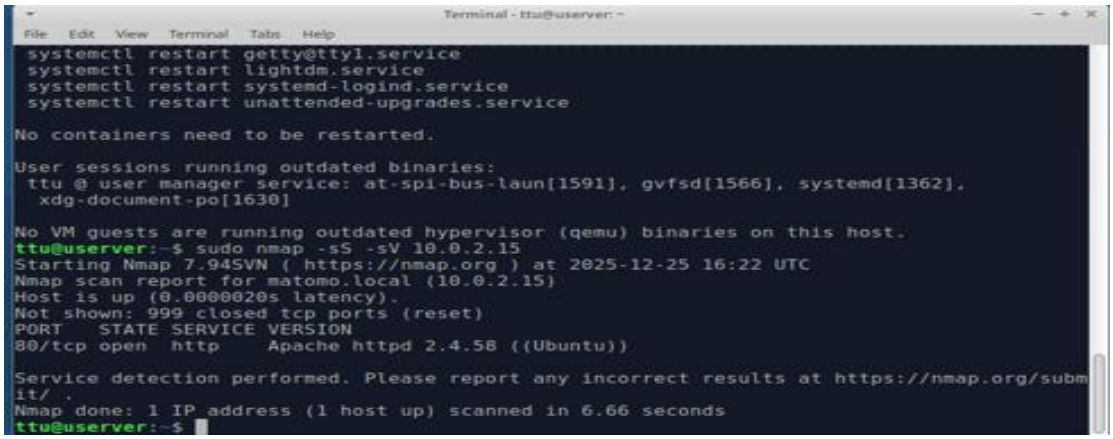
A non-intrusive penetration testing approach was followed, focusing on reconnaissance, enumeration, and vulnerability identification. No active exploitation was performed to avoid impacting system stability.

Tools Used:

- **Nmap** – Network and service enumeration
- **Nikto** – Web server vulnerability scanning
- **Manual Testing** – Direct access to sensitive files (robots.txt, server-status)

4.2 Network & Service Discovery using Nmap

An Nmap SYN scan with service and version detection was performed to identify open ports and running services on the target server (10.0.2.15).



Results :

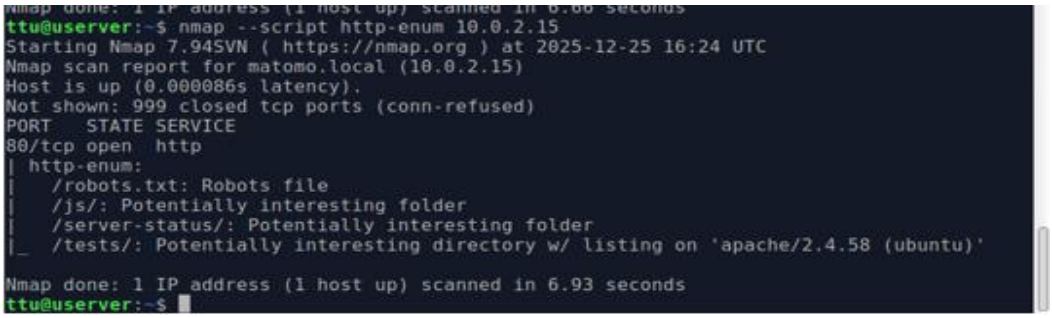
- Only **TCP port 80** was open
- Apache HTTP Server **2.4.58 (Ubuntu)** detected
- All other ports were closed

Security Assessment:

- **Risk Level:** Low
- **Assessment:** Reduced attack surface due to minimal exposed services

4.3 HTTP Enumeration and Directory Discovery

An HTTP enumeration scan was conducted using the Nmap Scripting Engine to identify accessible directories and files.



Results:

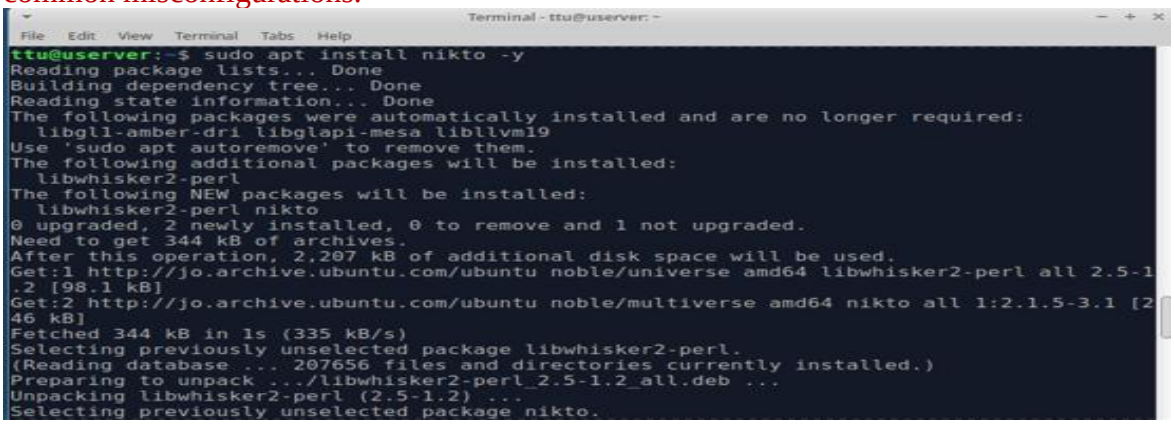
- `/robots.txt`
- `/server-status/`
- `/tests/`
- `/js/`

Finding	Risk Level
/server-status	High
/tests directory listing	Medium
robots.txt exposure	Low

4.4 Web Vulnerability Scanner Installation (Nikto)

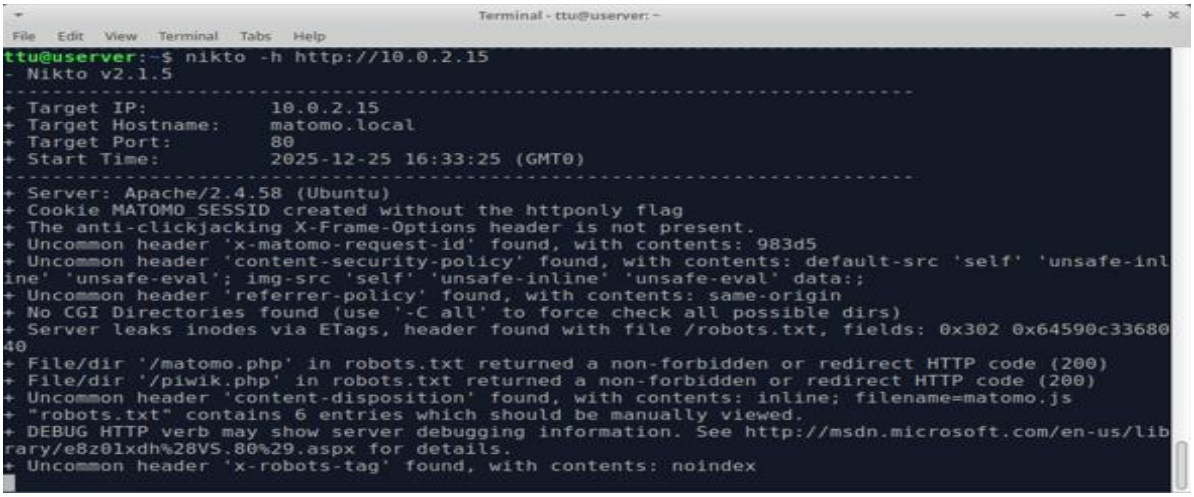
Nikto Installation

Nikto was installed to perform automated web server vulnerability scanning and identify common misconfigurations.



Nikto Web Vulnerability Scan Results

Nikto was used to scan the Matomo web application hosted on Apache to identify security weaknesses.



Results:

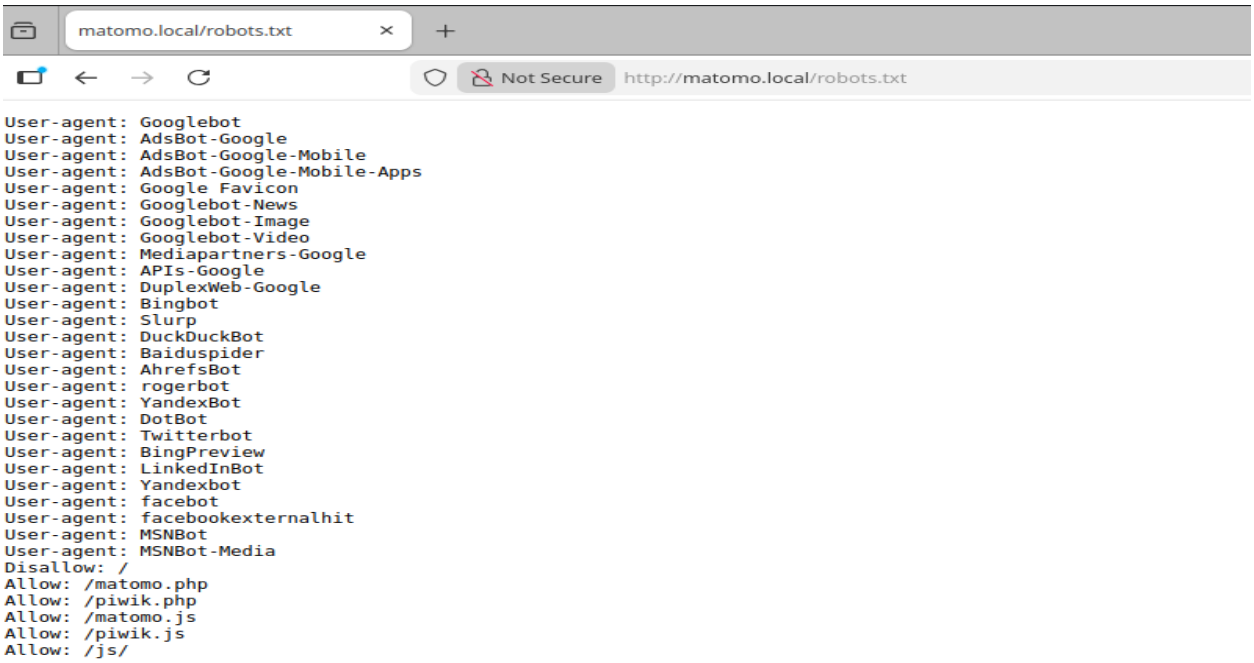
- Missing X-Frame-Options header
- Cookies without HttpOnly flag
- Apache version disclosure
- robots.txt exposure
- Weak Content Security Policy

Security Analysis:

These findings represent **common web server misconfigurations** that may increase exposure to client-side attacks and information disclosure but do not indicate critical exploitation paths.

Manual Web Inspection – robots.txt

The robots.txt file was manually accessed and revealed internal application paths such as `matomo.php` and `piwik.php`.



Vulnerability:

Information Disclosure via robots.txt

Explanation:

Exposed internal paths may assist attackers during reconnaissance.

Recommendation:

Sensitive application paths should not be listed in robots.txt in production environments.

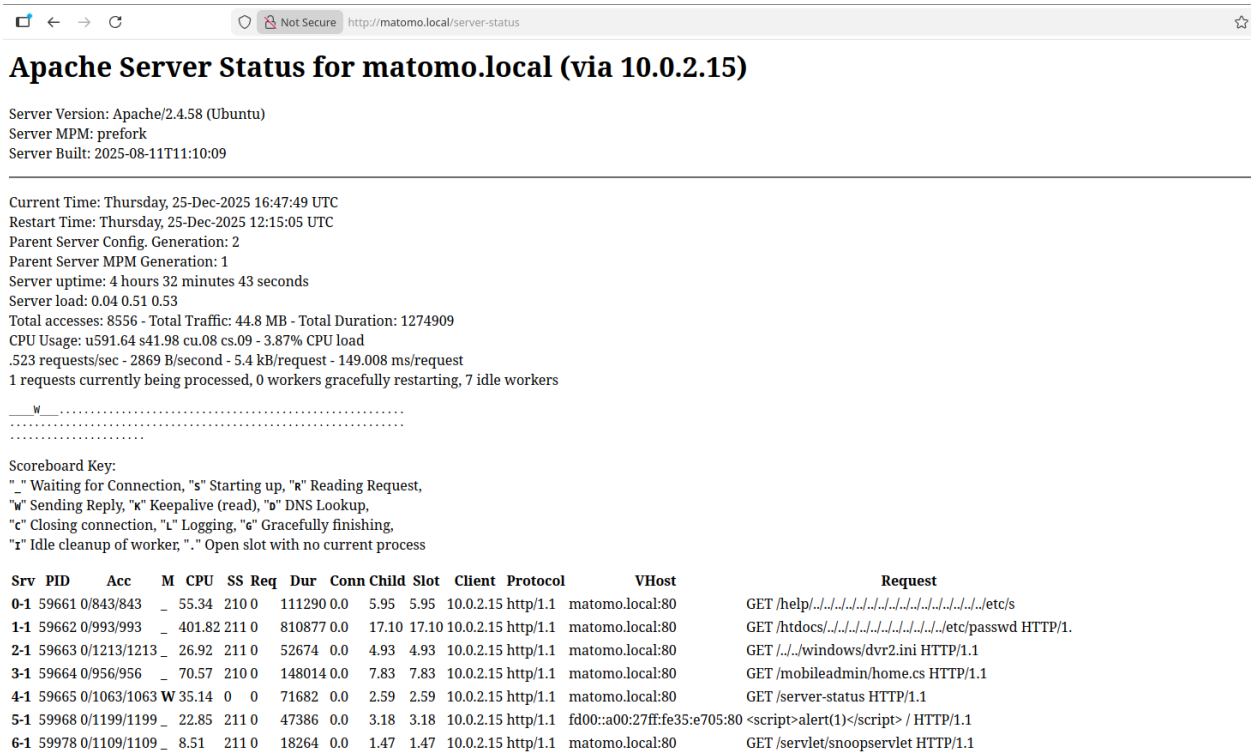
Exposed Apache Server-Status Page

Vulnerability Name:

Information Disclosure via Apache server-status

Severity:

High



The server-status page exposes:

- Apache version
- Server uptime
- Active requests
- Client IP addresses
- Worker processes

This information significantly aids attackers during reconnaissance.

Evidence:

- Apache/2.4.58 disclosed
- Internal request paths visible
- No authentication required

Recommendations:

- Disable server-status in production
- Restrict access to trusted IPs
- Enforce authentication

Final Recommendations

- Enable HTTP security headers (X-Frame-Options, CSP, HttpOnly)
- Disable or restrict Apache server-status
- Remove sensitive paths from robots.txt
- Follow security hardening best practices for Apache

Conclusion:

The conducted penetration testing activities identified several security misconfigurations mainly related to information disclosure and missing security headers. No critical vulnerabilities leading to system compromise were detected. The findings emphasize the importance of secure Apache configuration and continuous security monitoring to reduce the overall attack surface.